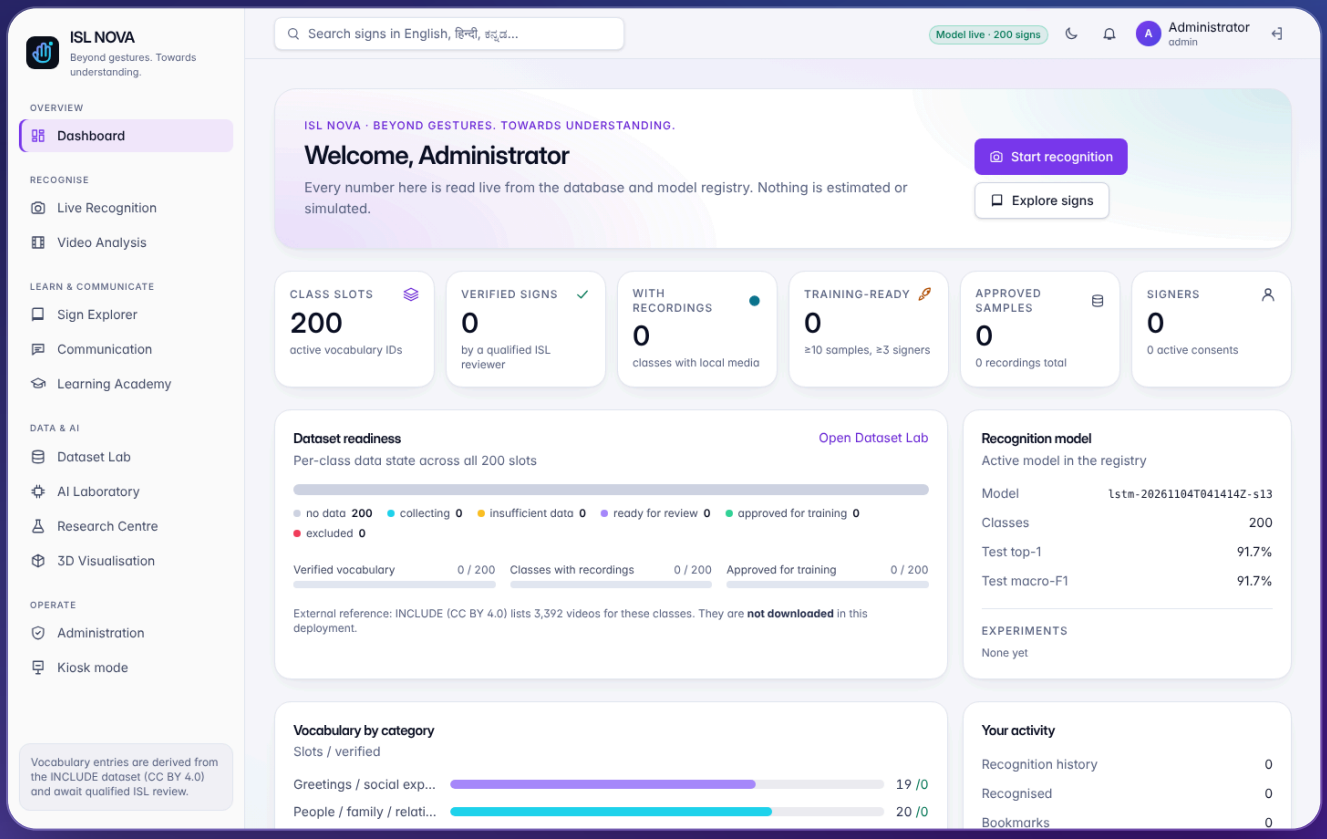
PROJECT REPORT · VERSION 0.1.0 · NOVEMBER 2026

ISL NOVA

Beyond Gestures. Towards Understanding.

An open, consent-first platform for **Indian Sign Language** recognition, learning and dataset building. Real-time recognition of **200 ISL signs** in the browser, a model trained on the genuine **INCLUDE** dataset, and every result traced, tested and stated honestly.



200
ISL sign classes trained

91.7 %
top-1 on 635 held-out test videos

3,370
real landmark sequences (INCLUDE)

12
application modules, all tested

github.com/nvaltryek-001/ISL-NOVA · branch feat/isl-nova-platform

Live demo: isl-nova.onrender.com

What is inside this report

01 **Introduction and problem statement** 3
Why ISL needs better tools, objectives, what we built

02 **System architecture and technology** 5
Modular monolith, browser vision, full technology stack

03 **Dataset: INCLUDE and the 200-sign vocabulary** 7
Source, licence, splits, categories, real dataset images, data pipeline

04 **Computer vision and feature engineering** 10
MediaPipe landmarks, the 32 × 204 feature contract

05 **Models and algorithms** 11
LSTM, GRU, logistic regression, DTW, calibration, rejection

06 **Results and evaluation** 13
Model comparison, confusion matrix, unknown signs, full pipeline

07 **The application and its interface** 17
12 modules, analytics dashboard, screenshots

08 **Testing, quality and security** 20
Test suites, CI, bugs found and fixed, security controls

09 **Deployment, setup and project structure** 21
Live demo, Docker, Windows scripts, folder structure, API

10 **Limitations, future work and conclusion** 23
What is not yet proven, next steps, references

The project at a glance

91.7 %
LSTM top-1 · 98.3 % top-5

0.900
unknown-sign AUROC

96
automated tests passing (60 + 17 + 19)

56.75 GB
of video processed, MD5-verified

4
model families compared

84
REST API operations

24 / 24
held-out samples via the live API

74.0 %
top-1 through the full browser pipeline

How to read this report. Every number in it was produced by a command that actually ran (GitHub Actions runs are cited by ID). Where something has *not* been measured, such as accuracy on new signers or a test with a real person at a webcam, the report says so plainly.

01 Introduction and problem statement

Why Indian Sign Language needs open, honest tools

India has millions of Deaf and hard-of-hearing people and very few certified ISL interpreters. Software cannot replace interpreters, but it can help people practise signs, look them up, and communicate simple messages. ISL NOVA is an end-to-end platform built to do that responsibly.

The problem

1 · Scarce, unconsented data

Few ISL datasets are openly licensed, and fewer record who signed and what they agreed to. Training on scraped videos is neither legal nor fair to the signers.

2 · Inflated accuracy claims

Many demos test on the same people they trained on. A high number then says little about how the system works for a new user.

3 · Systems that always guess

A plain classifier always returns some class, even for a sign it never saw, a half-visible hand or an empty frame.

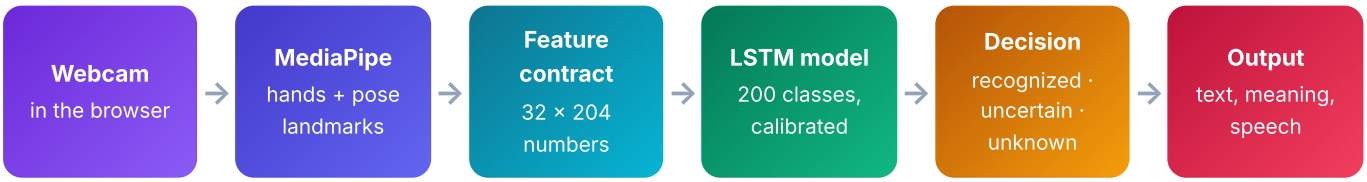
4 · Unverified vocabulary

Glosses copied from a dataset are labels, not reviewed ISL descriptions. Without expert review, a "sign dictionary" can mislead learners.

Objectives

1. **Recognise a 200-sign ISL vocabulary in real time** in the browser, with calibrated confidence and explicit rejection of unknown or poor input.
2. **Make every result traceable:** licensed source, verification state, data readiness, dataset version, model lineage and checksums.
3. **Collect data consent-first:** no recording without consent, no training without *model_training* consent, and withdrawal erases media.
4. **Evaluate honestly:** held-out tests evaluated once, signer-independent splits when signer IDs exist, open-set and new-signer workflows.
5. **Be usable by everyone:** learning, communication and kiosk modes, light and dark themes, phone to desktop, accessible by keyboard.

Our approach in one line



Only landmark coordinates leave the device. Video never does.

What we delivered

🎥 Real-time recognition

In-browser hand and pose tracking with automatic GPU→CPU fallback, capture-one-sign and continuous modes, top-5 answers, speech output.

🧠 A real trained model

LSTM trained on 2,448 genuine INCLUDE recordings for all 200 classes; 91.7 % top-1 on the official test split.

🗂️ 200-sign vocabulary

ISL001–ISL200 with licensed references, verification and readiness states, English/Hindi/Kannada meanings (pending where unknown).

🔒 Consent-first data lab

Pseudonymous signers, consent scopes, recording studio, four-eyes review, immutable dataset versions, erasure on withdrawal.

📊 AI laboratory

Held-out metrics, 200 × 200 confusion matrix, model registry with promote/rollback, experiments, new-signer evaluation.

✅ Expert verification

Only the linguist role can verify a sign, after documenting it and ticking a three-item checklist enforced by the API.

🎓 Learning and communication

Lessons and quizzes built only from verified signs; a word-by-word message composer with speech.

☁️ Deployed and reproducible

Docker image, free-tier live demo with the model installed at start-up, Windows one-command setup, GitHub Actions pipelines.

🧪 Tested end to end

96 automated tests on Linux and Windows, PostgreSQL and SQLite, plus evaluation runs on real held-out data.

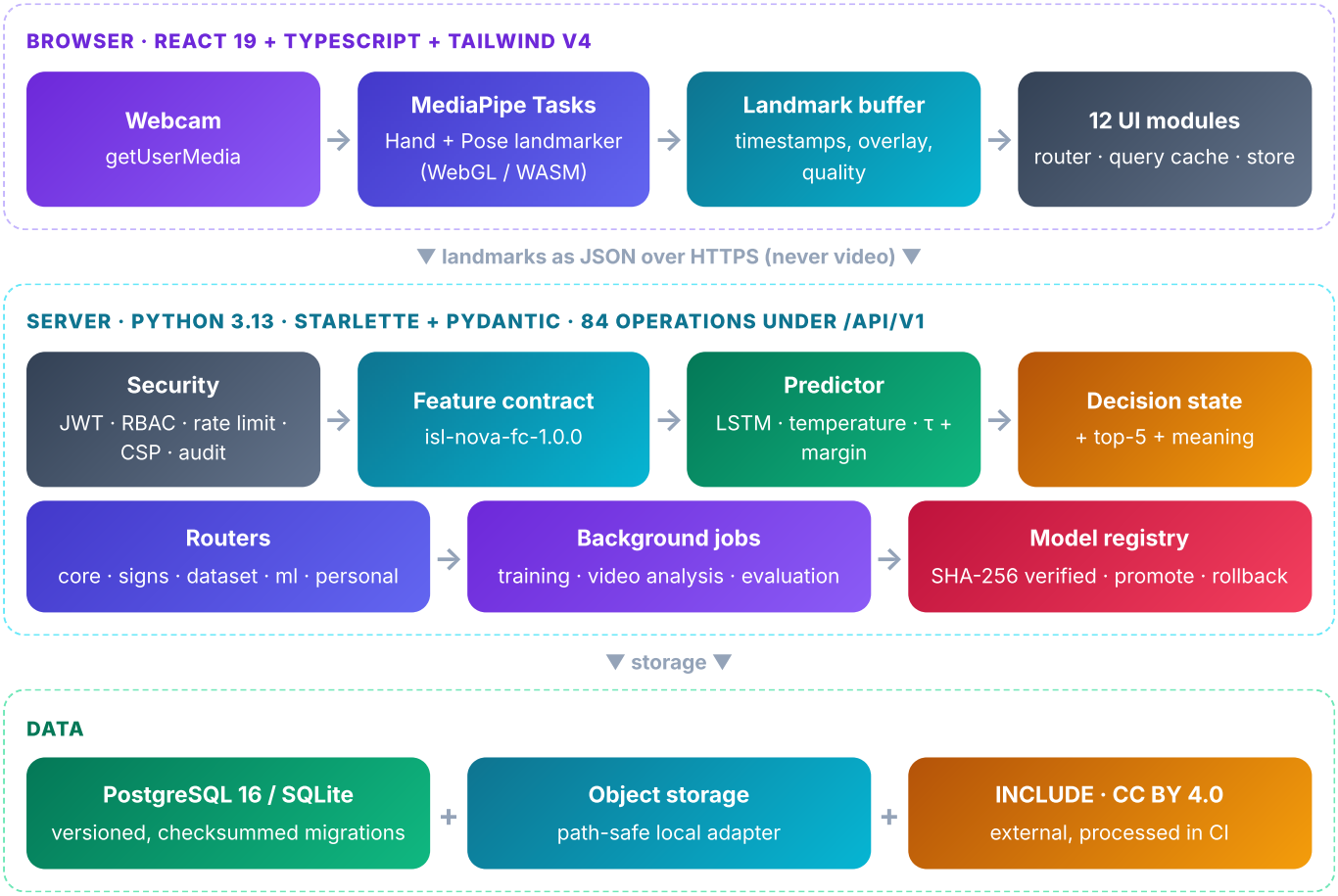
Project journey

Stage	What happened	Evidence
1 · Audit & design	Repository audit, specification, architecture, 200-slot vocabulary built from INCLUDE's class list	docs/, dataset/
2 · Platform	API, database, auth/RBAC, 12 web modules, consent pipeline, ML package with gradient-checked LSTM/GRU	CI green
3 · Data	44 Zenodo parts (56.75 GB) downloaded and MD5-checked on GitHub runners; MediaPipe extraction; raw video deleted	run 41764539272
4 · Training	LSTM, GRU, logistic regression, DTW trained in parallel; best chosen on validation; published as a release	run 41771084083
5 · Evaluation	Unknown-sign test (62 extra glosses), raw-video check, virtual-camera replay through the real UI	runs 41815223675, 41860050670
6 · Delivery	Render deployment, smoke tests against the live site, Windows scripts, local training, documentation	runs 41861458853, 41961463214

02 System architecture and technology

One Python service, one React app, one shared ML package

ISL NOVA is a **modular monolith**: a single Python process serves the JSON API and the built web client, and a shared ML package implements the feature contract, training and inference. This keeps deployment simple (one container) while modules stay cleanly separated.



Design principles

Privacy by architecture

Vision runs in the browser; the server only ever sees landmark coordinates for live recognition.

One feature contract

The same 204 features are computed in Python and TypeScript; a parity test keeps them within 2×10^{-5}.

Honest by default

Five explicit answer states; synthetic models are flagged and can never be promoted in production.

Technologies used

Layer	Technology	Role in ISL NOVA
Frontend	React 19.2, TypeScript 6 (strict), Tailwind CSS 4.3, esbuild 0.28	12 modules, design system with light/dark tokens, in-house router, query cache and store, SVG charts
Computer vision	Google MediaPipe Tasks Vision 0.10.14 (browser), MediaPipe 0.10.32 (Python), OpenCV 5	21-point hand landmarks ×2 and 33-point pose; video decoding, probing and server-side extraction
Backend	Python 3.13, Starlette 1.7, Pydantic 2.13, Uvicorn, PyJWT, PBKDF2	Typed routing with generated OpenAPI, validation, JWT auth, five roles, rate limits, structured logs
Database	PostgreSQL 16 (production), SQLite (development, demo)	28 tables, versioned checksummed migrations, transactions, audit log
Machine learning	NumPy (LSTM/GRU with hand-written backprop), scikit-learn, batched DTW	Training, temperature scaling, rejection threshold, registry, inference
Testing	pytest, node:test, Playwright (Chromium), Ruff	Unit, API, CRUD, RBAC, parity, end-to-end browser tests, lint
DevOps	GitHub Actions (Linux + Windows), Docker, Render (free plan)	CI, staged data/train/evaluation pipelines, model releases, live demo
Local setup	PowerShell scripts, Python helpers	One-command Windows setup, start, test, training and prediction check

Libraries and models list

Pretrained vision models (Google MediaPipe)

- **hand_landmarker.task** (float16): palm detector + 21-keypoint hand model
- **pose_landmarker_lite.task** (float16): 33-keypoint body pose
- Delegates: GPU (WebGL) with automatic CPU (WASM / XNNPACK) fallback

Models trained in this project

- **LSTM**: 1 layer, hidden 128, 221,896 parameters (selected)
- **GRU**: same shape, about 179k parameters
- **Logistic regression** on pooled statistics (scikit-learn)
- **DTW 1-NN**: nearest neighbour under dynamic time warping

Why these choices

Landmarks, not pixels

Pose landmarks are small, privacy-friendly and robust to background and clothing, so a compact model trains on CPU in minutes.

NumPy models

No deep-learning framework was installable in the build sandbox, so LSTM/GRU were implemented directly and gradient-checked.

Starlette + esbuild

The tested foundations of FastAPI and Vite; routes and bundling port 1:1 if the project later adopts those tools.

All dependency versions are pinned (backend/requirements.txt, frontend/package.json). A CI audit job runs pip-audit (no known vulnerabilities) and npm audit; the one advisory it found (zod) was fixed by removing the dependency.

03 Dataset: INCLUDE and the 200-sign vocabulary

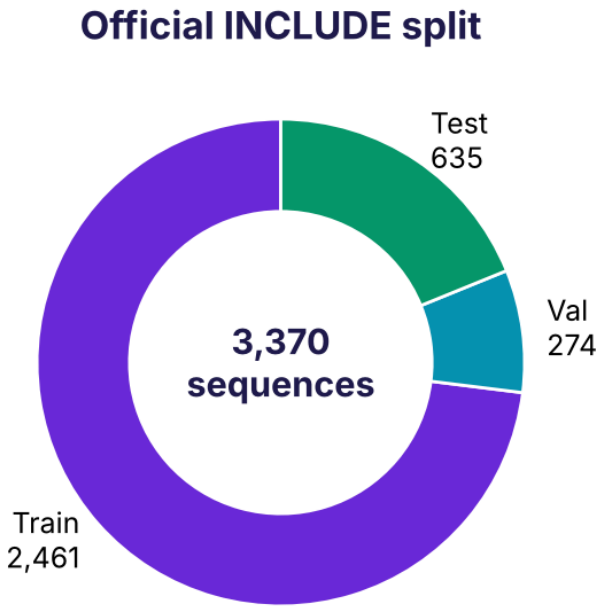
Real, licensed recordings, processed reproducibly

INCLUDE at a glance

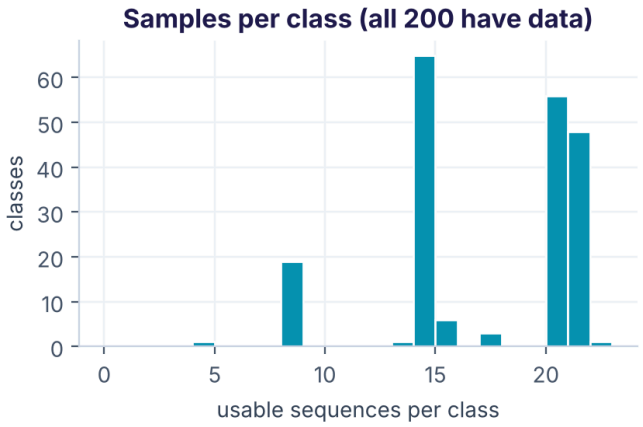
INCLUDE is a large isolated-sign ISL dataset by A. Sridhar, R. G. Ganesan, P. Kumar and M. Khapra (AI4Bharat, IIT Madras), published at ACM Multimedia 2020 and released on Zenodo (record 4010759) under **CC BY 4.0**. It contains 263 word signs recorded in 1920 × 1080 at 25 fps.

ISL NOVA uses 200 of those glosses as its vocabulary (**ISL001–ISL200**). The remaining 63 are held in reserve, and 62 of them now serve as the **unknown-sign** test set.

Metric	Value
Zenodo parts downloaded (MD5-verified)	44 / 44
Data volume	56.75 GB
Videos processed	4,284
Usable sequences, 200 classes	**3,370**
Failed videos / insufficient input	0 / 15
Classes with train + val + test	189 / 200
Out-of-vocabulary sequences (62 glosses)	887
Linguistically verified signs	0 / 200



INCLUDE's official train / validation / test lists are kept exactly.



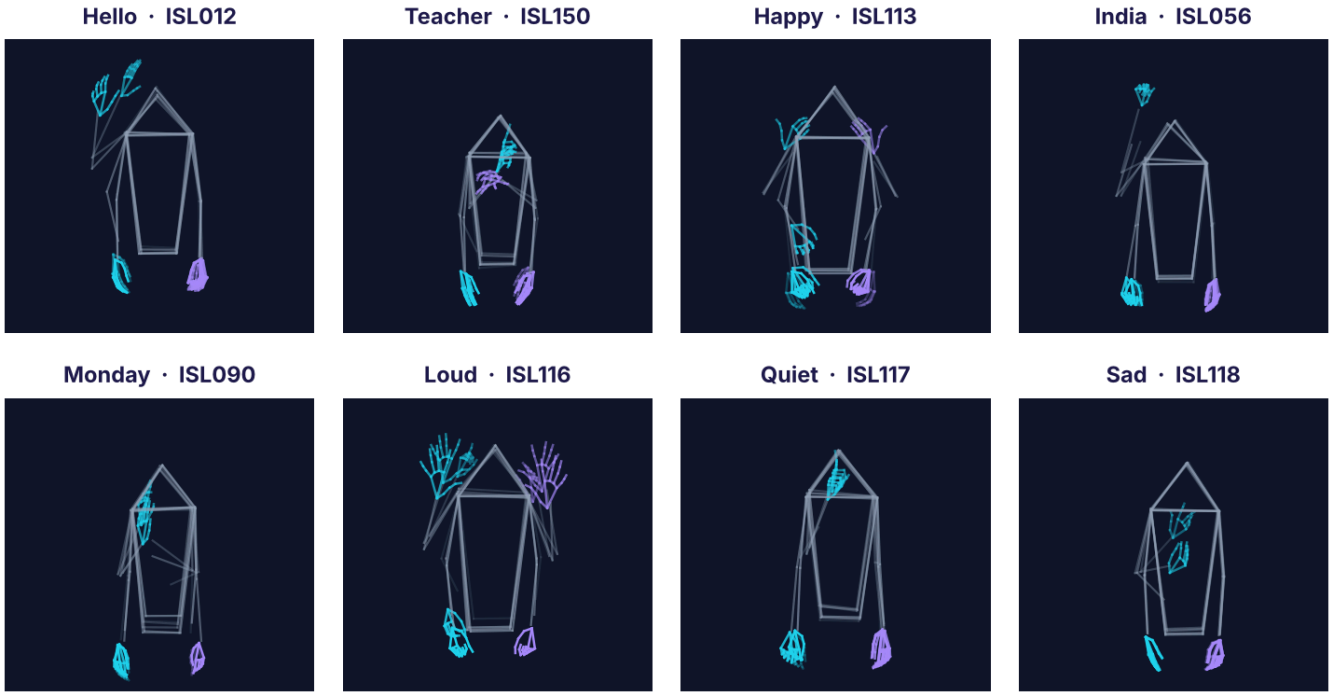
Every one of the 200 classes has real recordings: median 20, range 4–22.

> INCLUDE publishes **no signer IDs**, so test signers may also appear in training. Results on this split are therefore **not signer-independent**; the platform includes a separate new-signer workflow for that.

Real dataset images

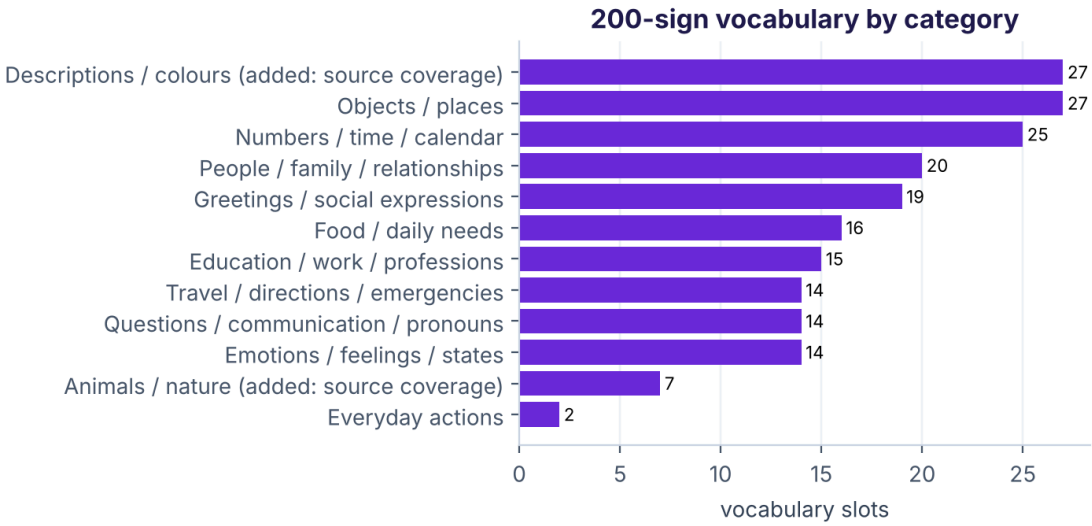
Below are genuine INCLUDE **test** recordings, shown as the landmarks our pipeline extracted (raw videos are not redistributed; landmarks are what the model actually sees). Cyan is the right hand, violet the left hand, grey the upper body; six frames are overlaid so the movement is visible.

Real INCLUDE test recordings as extracted landmarks (6 frames overlaid, later = brighter)



Eight held-out test recordings from INCLUDE (CC BY 4.0, Sridhar et al. 2020), rendered from the processed landmark files.

The 200-sign vocabulary



Vocabulary slots by planning category. INCLUDE has few everyday verbs and no food or question words, which new recordings must fill.

Verification states

proposed → reference_found → review_pending → **verified** | rejected. Only the linguist role verifies.

Readiness states

no_data → collecting → ready_for_review → approved_for_training. Needs a verified sign, ≥ 10 recordings, ≥ 3 signers.

Meanings

English from the source; Hindi and Kannada left *pending* rather than invented.

How the data was processed

The 56.75 GB of video never touched a laptop or the Git repository. A staged GitHub Actions workflow processed it in parallel and kept only landmarks.

Preflight disk, RAM, Zenodo record, licence, label map → **Download** 44 parts in parallel, MD5 + retries → **Match** official split lists, dedupe → **Extract** MediaPipe hands + pose, downscale 960 → **Delete video** keep landmarks only → **Merge** manifest, per-class counts, failures

Integrity checks

- ✔ MD5 of every Zenodo part, SHA-256 of every landmark file and source video
- ✔ Duplicate recording IDs fail the merge
- ✔ Failures and insufficient-input sequences recorded, not hidden
- ✔ Monotonic MediaPipe timestamps across videos

Outputs

- ✔ **include-official-v1**: 3,370 sequences, 176 MB, published as release `dataset-include-landmarks-v1`
- ✔ **include-official-v2**: adds 887 out-of-vocabulary sequences for open-set tests
- ✔ Attribution file shipped inside the archive (CC BY 4.0)

Consent-first collection inside the app

Signer pseudonymous ID → **Consent** scopes, retention → **Session** lighting, distance → **Record** video + landmarks, quality checks → **Annotate** person A → **Approve** a different person B → **Freeze** dataset version

No consent, no data

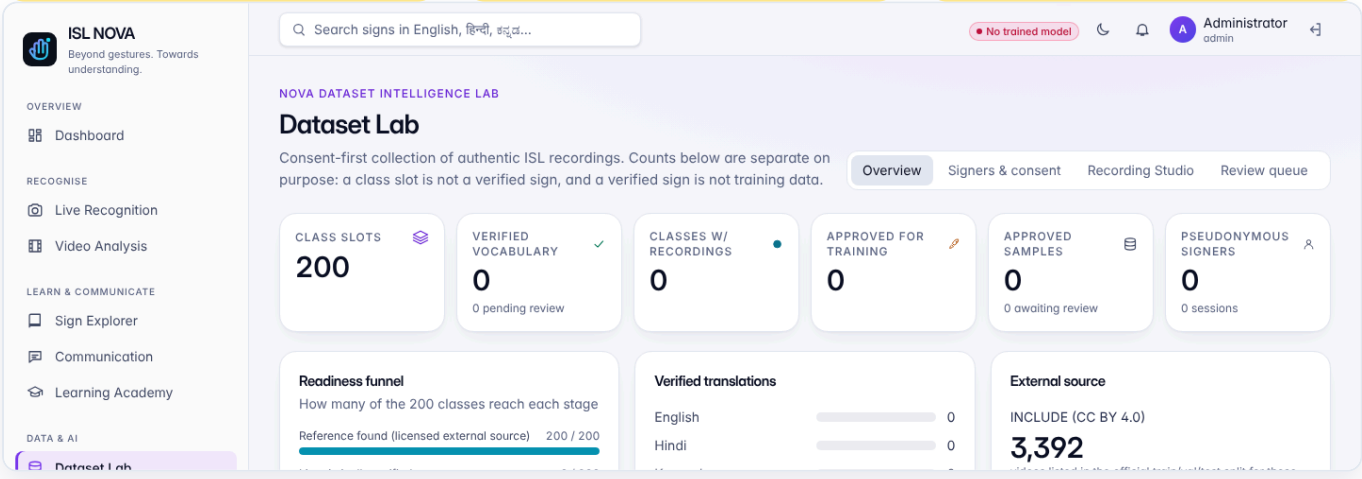
Recording is impossible without a consent record; training requires the model_training scope.

Withdrawal = erasure

Revoking consent deletes the signer's media and excludes it from future versions.

Four-eyes review

Annotator and approver must be different people; every media access is audited.



Dataset Intelligence Lab: readiness funnel and per-class table driven by live database counts.

04 Computer vision and feature engineering

From camera pixels to 32 × 204 numbers

Landmark extraction with MediaPipe

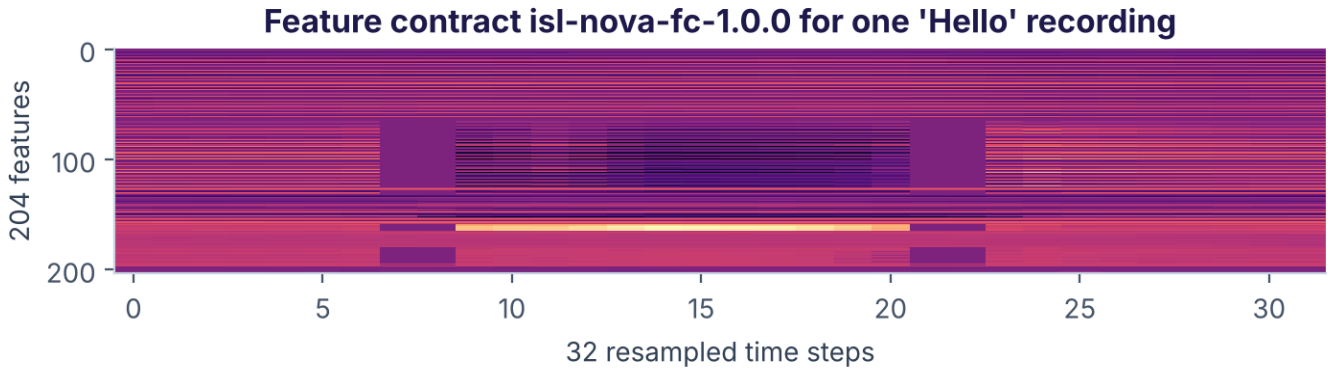
2 × 21
hand keypoints (x, y, z) per frame

33
pose keypoints with visibility

~10–30 fps
in the browser (CPU / GPU)

Frames are processed **unmirrored** so that left and right hands match between browser and server; only the preview is mirrored for the user. If the GPU delegate runs below 5 fps for three seconds (software WebGL, weak GPUs), the app switches to the CPU delegate automatically and shows a badge.

The feature contract `isl-nova-fc-1.0.0`



One real INCLUDE recording after the contract: 32 time steps × 204 features (colour = value).

Feature group	What it captures	Dims
Hand shape	21 landmarks per hand, relative to the wrist and scaled by hand size	2 × 63
Hand position	wrists relative to the shoulders and torso, scaled by shoulder width	≈ 6
Upper body	pose keypoints relative to the body centre	≈ 30
Two-hand relation	distances between the hands and fingertips	≈ 10
Joint angles	finger flexion angles	≈ 20
Presence + motion	hand-present flags and wrist velocity	≈ 8
Total per frame, resampled to 32 frames		**204**

Quality gate

Too few frames or hands visible in too few frames → **insufficient_input** with a human-readable reason, instead of a guess.

Parity

The TypeScript port is tested against Python fixtures (two hands, one hand, gaps, no pose) to within 2×10^{-5}.

Group sizes are approximate; the authoritative list of all 204 named features is generated from code into docs/FEATURE_CONTRACT.md.

05 Models and algorithms

Four model families, one honest decision rule

LSTM: Long Short-Term Memory selected

A recurrent network that reads the 32 frames in order and keeps a memory of the movement. Final and mean-pooled hidden states feed a 200-way softmax.

- 1 layer, hidden size 128, 221,896 parameters, 1.78 MB
- Adam, lr 0.002, weight decay 1e-4, batch 32
- Augmentation, inverse-√frequency class weights
- Early stopping on validation (stopped at epoch 49)

GRU: Gated Recurrent Unit

A lighter recurrent network with update and reset gates instead of the LSTM's three gates and cell state.

- Same shape and recipe as the LSTM
- About 179k parameters, 1.44 MB
- Fastest recurrent model (0.17 ms / sample)

Logistic regression on pooled statistics

Each feature's mean, standard deviation, minimum and maximum over time form one vector, classified linearly (scikit-learn).

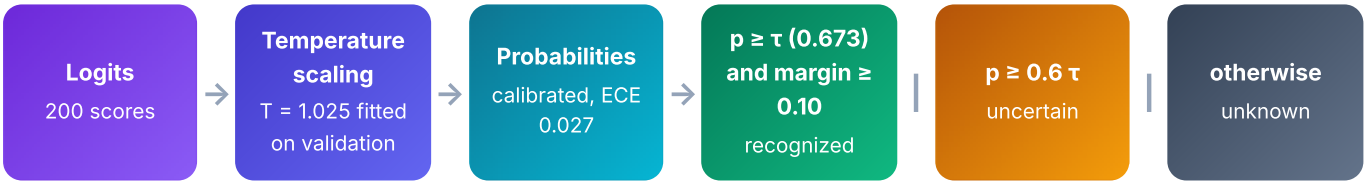
- No temporal order, but very strong baseline
- 0.02 ms / sample, 0.99 MB

DTW 1-NN: Dynamic Time Warping

Finds the training sequence that best aligns with the query when time is stretched or compressed, and copies its label.

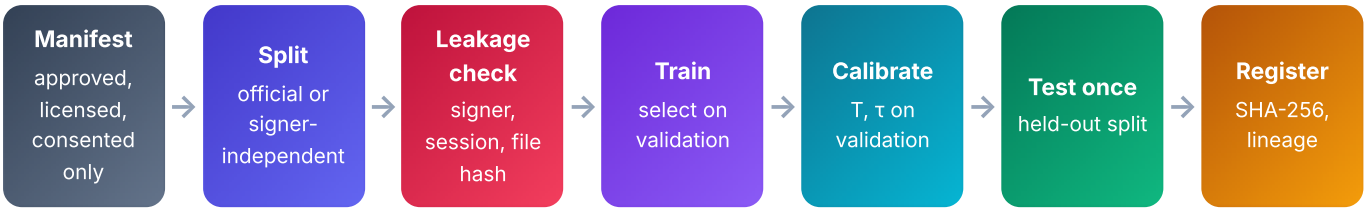
- Batched NumPy DTW with a Sakoe–Chiba window
- No training, but 130 ms / sample, 64 MB

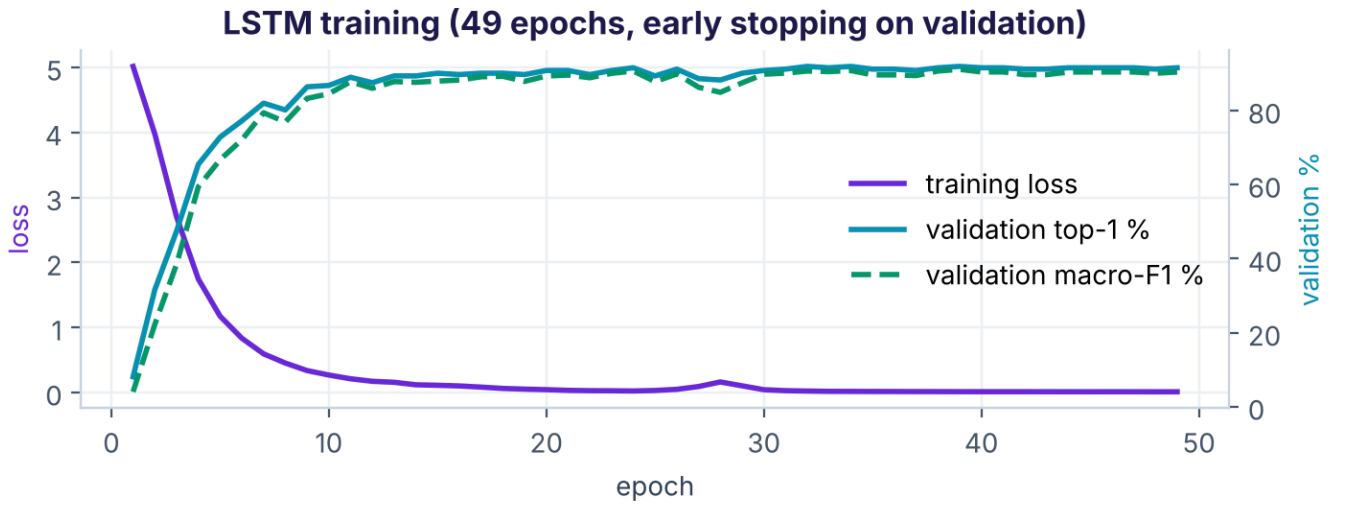
From scores to an honest answer



τ is chosen on validation so that 95 % of correct validation predictions are kept. The test split never influences any choice. Two further states, **insufficient_input** and **model_unavailable**, are returned when the input or the server cannot support a prediction.

Training pipeline





Genuine training log of the released LSTM: loss falls steadily while validation accuracy and macro-F1 climb above 90 %.

What every trained model records

- Feature-contract version and dataset version
- Manifest SHA-256 and the recording IDs per split
- Leakage report and excluded recordings
- Top-1/5, macro and weighted F1, per-class report, confusion matrix, ECE, open-set AUROC, latency, size

Registry rules

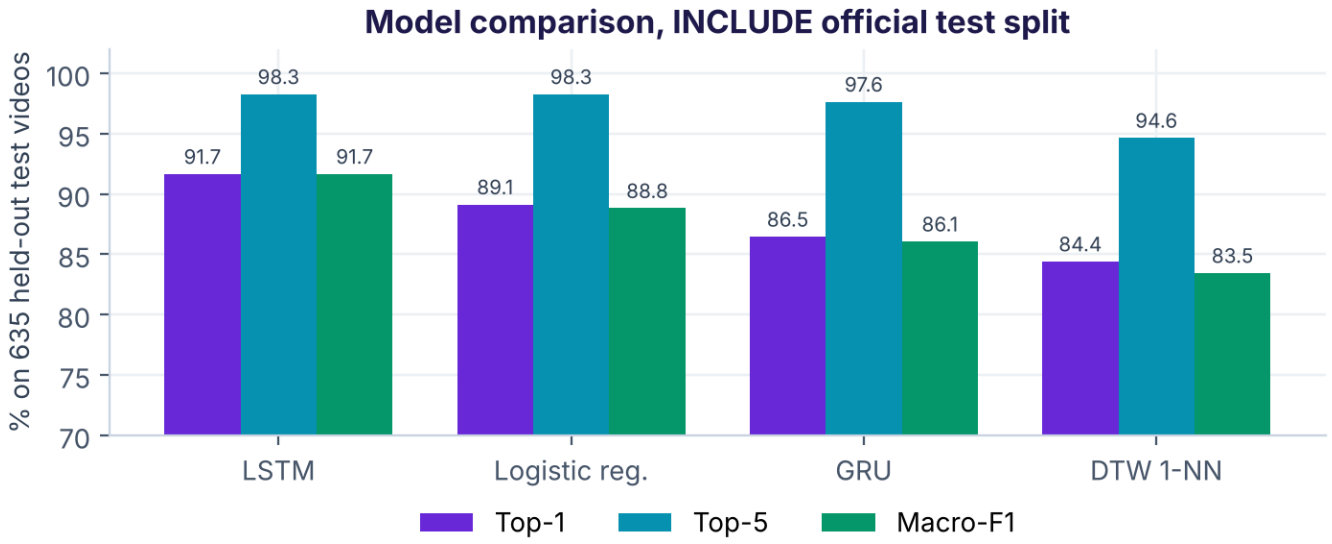
- Weights hash verified at every load
- Promotion and rollback by admins, audited
- Synthetic models refused in production
- Released model: `model-lstm-20261104T041414Z-s13`, SHA-256 `e0979cd5…c543c0`

Where training ran

Environment	What	Result
GitHub Actions, 4 vCPU (run 41771084083)	All four models in parallel, LSTM in about 8.5 min	LSTM 91.7 % top-1
Windows runner (run 41961463214)	`train_windows.ps1 -Model pooled-logreg` on the published landmarks	89.1 %, identical to CI
Any laptop	`python scripts/train_local.py --model all`	same recipe, small numeric differences

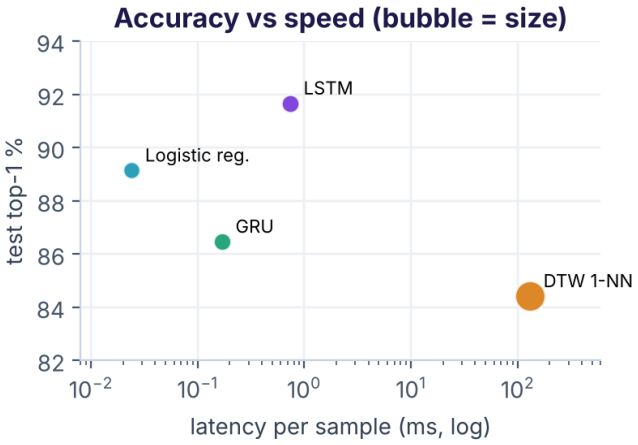
06 Results and evaluation

Genuine held-out results, evaluated once



All four models on the 635-video INCLUDE test split (training run 41771084083, seed 13).

Model	Val macro-F1	Top-1	Top-5	Precision	Recall	Macro-F1	Weighted-F1	ECE	ms / sample	Size
LSTM ★	**91.1 %**	**91.7 %**	**98.3 %**	**93.0 %**	**92.4 %**	**91.7 %**	**91.5 %**	**0.027**	**0.73**	**1.78 MB**
Logistic regression	88.1 %	89.1 %	98.3 %	90.4 %	89.9 %	88.8 %	88.9 %	0.024	0.02	0.99 MB
GRU	86.7 %	86.5 %	97.6 %	88.0 %	87.4 %	86.1 %	85.8 %	0.017	0.17	1.44 MB
DTW 1-NN	80.7 %	84.4 %	94.6 %	85.7 %	84.8 %	83.5 %	83.8 %	0.040	130	64 MB

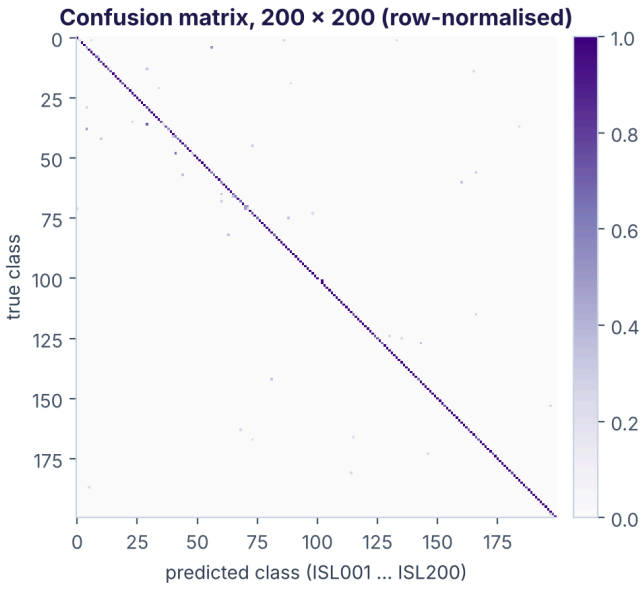


The LSTM gives the best accuracy at sub-millisecond model latency.

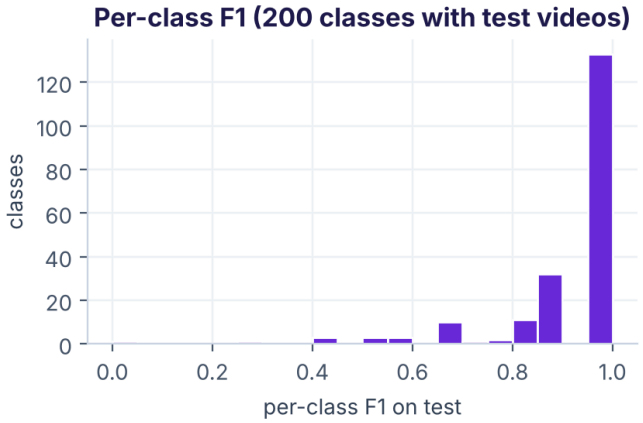
Key findings

- ✔ LSTM wins on validation and on test, so model selection was not lucky.
- ✔ Top-5 is 98.3 %: the right sign is almost always among the alternatives shown.
- ✔ Calibration is good (ECE 0.027): a 90 % confidence means about 90 % correct.
- ✔ Re-running all 635 tests through the production `Predictor` gave 91.65 %, 24 ms median per call.

> Single seed. Differences of a few points between models can come from seed variation.



A strong diagonal across all 200 classes: most signs are rarely confused.



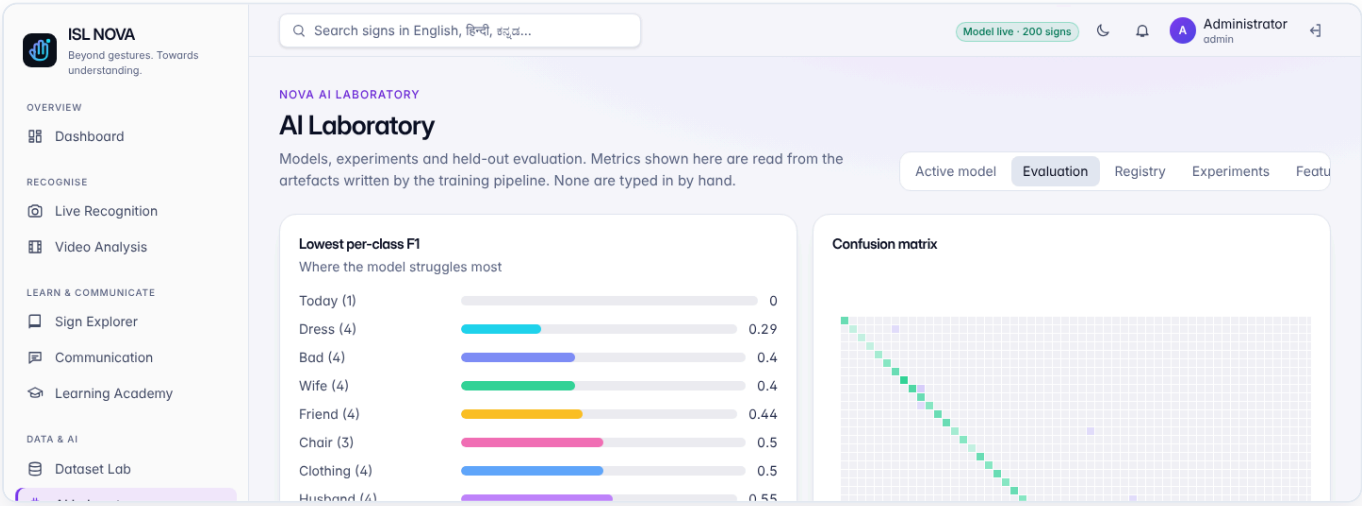
133 of 200 classes reach F1 ≥ 0.9; a small tail of hard classes remains.

Hardest classes on test

Sign	F1	Precision	Recall	Test videos
Today	0.00	0.00	0.00	1
Dress	0.29	0.33	0.25	4
Bad	0.40	1.00	0.25	4
Wife	0.40	1.00	0.25	4
Friend	0.44	0.40	0.50	4
Chair	0.50	1.00	0.33	3

> With only 1–4 test videos per class, per-class scores carry wide uncertainty and should not be over-read.

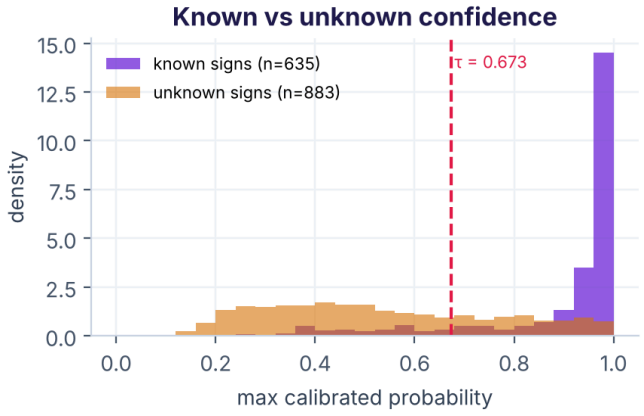
The evaluation dashboard



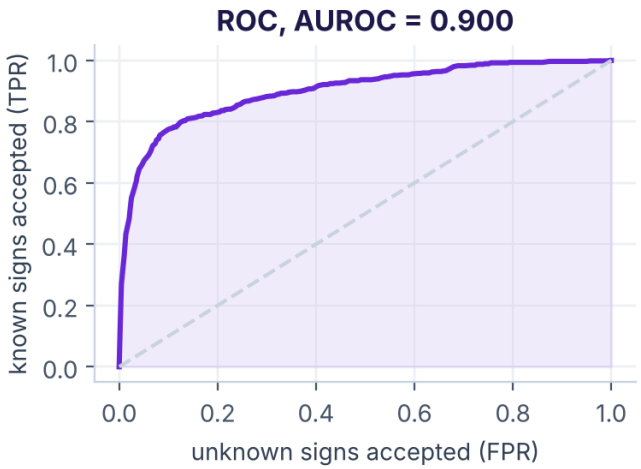
The AI Laboratory's evaluation view renders the same per-class F1 and confusion matrix from the live model files.

Can it say "I don't know"?

The model, unchanged, was given **887 recordings of 62 INCLUDE signs that are not in its vocabulary** (run 41815223675). They were never used for training, calibration or choosing the threshold. A good system should answer *uncertain* or *unknown* for them.



Known signs cluster near 1.0; unknown signs spread widely, but many still pass τ.



ROC of the max-probability score: AUROC 0.900.

0.900
AUROC known vs unknown

29.3 %
false acceptance (unknown shown as known)

12.1 %
false rejection of known signs

89.1 / 70.7 %
unknown-detection precision / recall

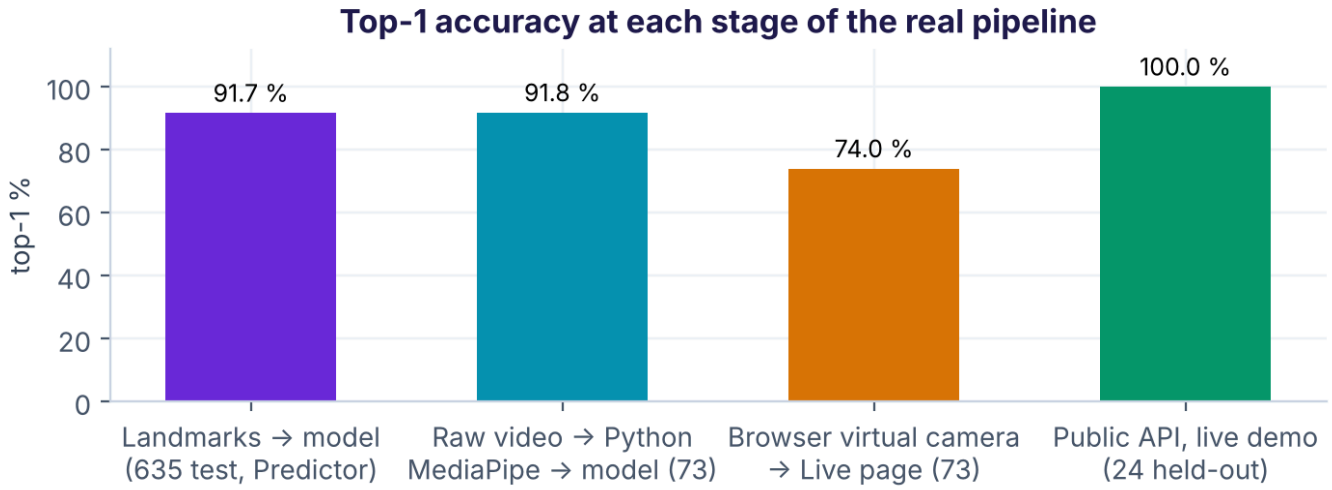
Threshold trade-off

Threshold	0.40	0.50	0.60	0.70	0.80	0.90
Known signs accepted	96.7 %	93.7 %	89.9 %	86.8 %	82.4 %	75.3 %
Unknown signs rejected	33.5 %	49.9 %	63.2 %	73.6 %	82.7 %	91.8 %

Honest reading. About 3 in 10 unseen signs are still confidently mapped to a known sign. So the UI always shows the top-5 alternatives and the state, and never speaks a result automatically. Better detectors (energy scores, a background class, embedding distance) are future work.

Accuracy at every stage

A model score on stored landmarks is only the first step. We measured the system at each stage a real user's sign passes through.

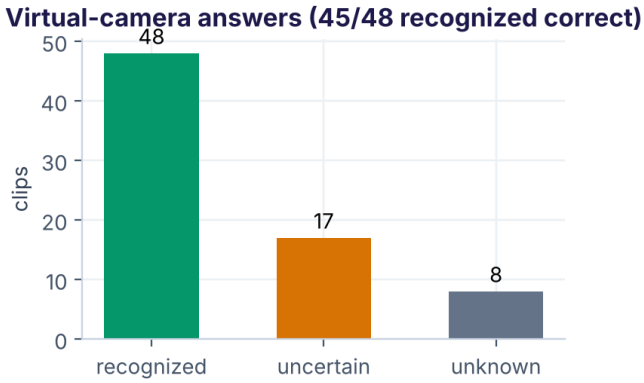


From stored landmarks to the full browser pipeline (runs 41771084083, 41774486592, 41860050670, 41861458853).

Virtual-camera replay through the Live page

73 held-out INCLUDE test clips were played into headless Chromium **as its webcam**. Each went through in-browser MediaPipe, the capture button, the API and the model, exactly like a user.

Metric	Value
Clips completed	73 / 73
Top-1 / top-5	**74.0 %** / 89.0 %
"Recognized" answers that were correct	**45 / 48**
Same clips, offline Python	91.8 %
Hand-tracking rate	~10 fps (CPU)



When the system says "recognized", it is right 94 % of the time.

The gap to 91.8 % comes mainly from frame rate: the CI runner had no GPU, so MediaPipe ran at ~10 fps instead of the clips' 25 fps. This is a replay of recordings, **not a live person**.

Live demo API

24 / 24 held-out samples recognized correctly through isl-nova.onrender.com.

On Windows

24 / 24 through a locally started server, on the runner and on the owner's PC.

Not yet measured

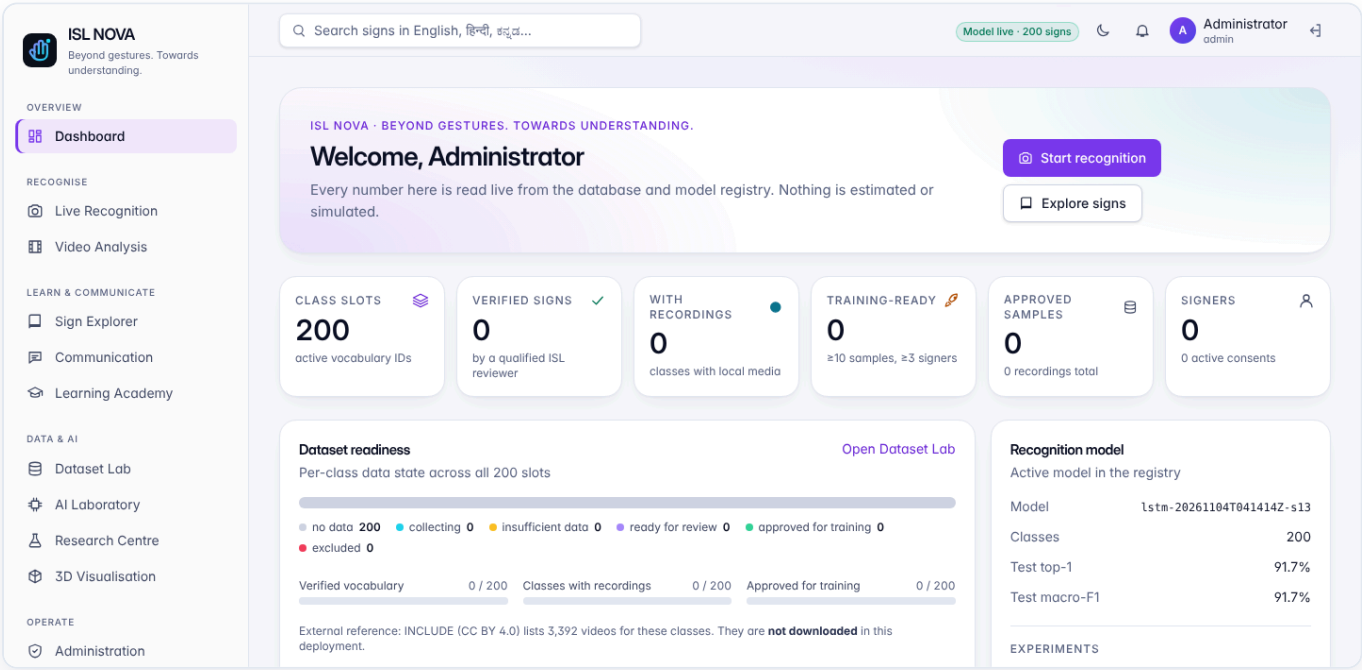
A real person at a webcam, and signers outside INCLUDE.

07 The application and its interface

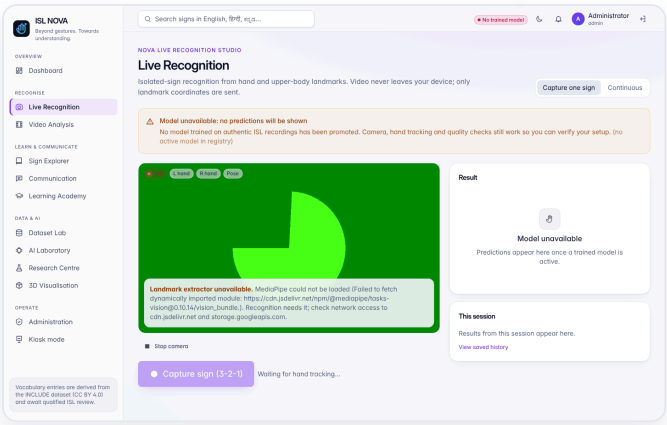
Twelve modules, one design system, light and dark

#	Module	What it does
1	**Analytics Dashboard**	Live counts from the database and registry: slots, verified signs, recordings, training-ready classes, model status, system health
2	**Live Recognition Studio**	Camera selection, landmark overlay, capture one sign or continuous window, fps badge, top-5, meanings, speech, corrections
3	**Video Analysis**	Validated upload, background hand tracking, timeline, windowed inference, JSON export, 24-hour auto-deletion
4	**Sign Explorer**	Multilingual search, filters, licensed references, bookmarks, full CRUD for reviewers, verification checklist
5	**Communication Workspace**	Word-by-word composer from sign meanings, editable text, speech, saved phrases
6	**Learning Academy**	Lessons by category, quizzes from verified signs only, progress tracking
7	**Dataset Intelligence Lab**	Readiness funnel, signers, consent, Recording Studio, four-eyes review, validation, dataset versions
8	**AI Laboratory**	Held-out metrics, confusion matrix, registry, experiments, new-signer evaluation, feature-contract viewer
9	**Research Centre**	Research questions, protocols, verified references
10	**3D Landmark Viewer**	Orbit and step through real landmark sequences and wrist trajectories
11	**Admin**	User roles, feedback inbox, audit log
12	**Kiosk Mode**	Full-screen, high contrast, large targets, quick phrases, one-tap reset

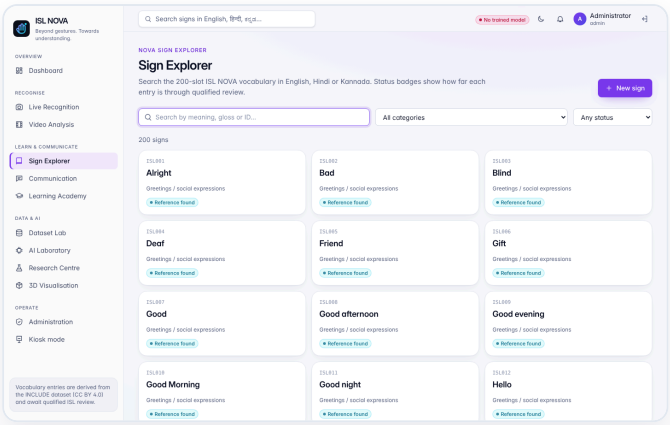
Analytics dashboard interface



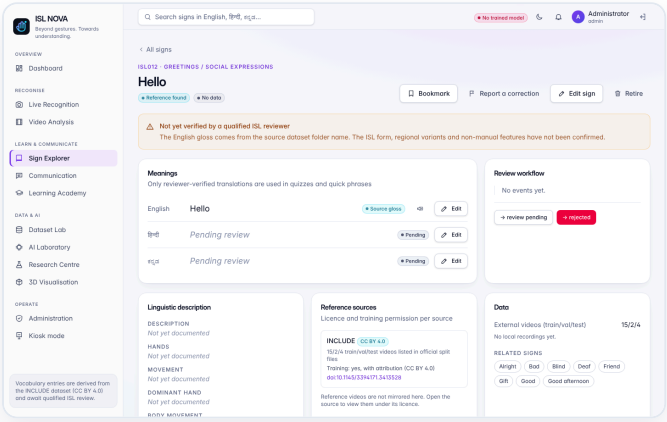
Dashboard with the trained 200-class model live: every number comes from the database or the model registry, never hard-coded.



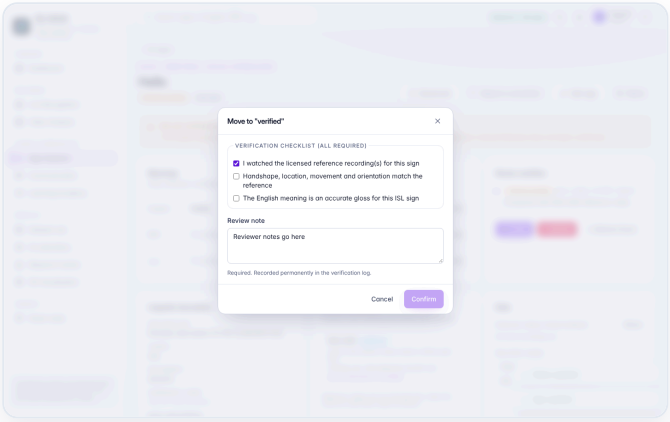
Live Recognition. Camera, landmark badges, capture controls and states (captured offline, before the model was installed).



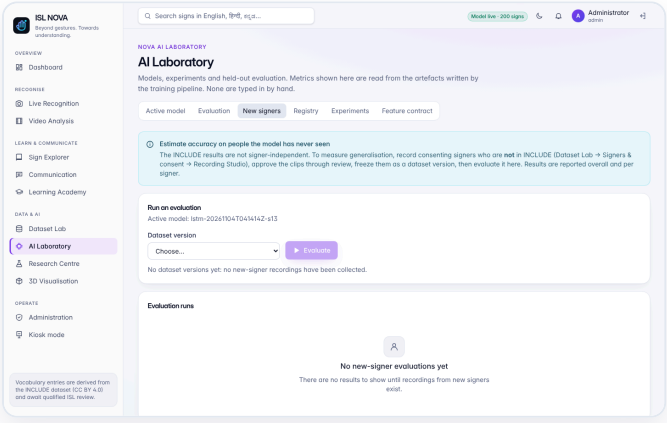
Sign Explorer. All 200 signs with status and readiness filters.



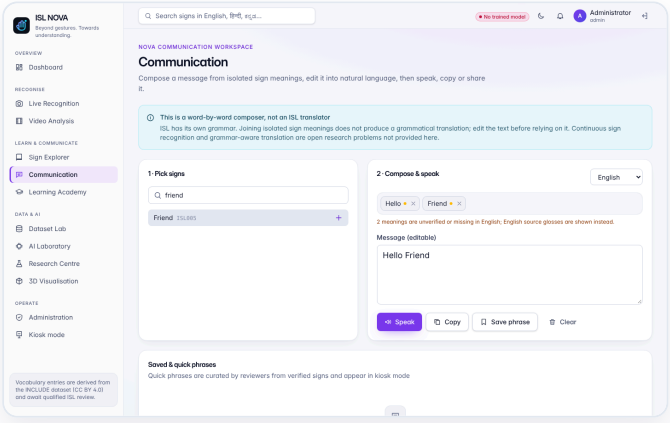
Sign detail. Licensed source, pending meanings and the review log.



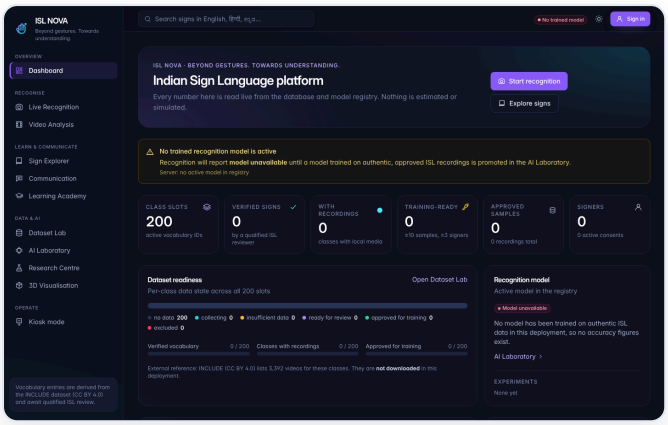
Verification. Confirm stays disabled until all three checks are ticked.



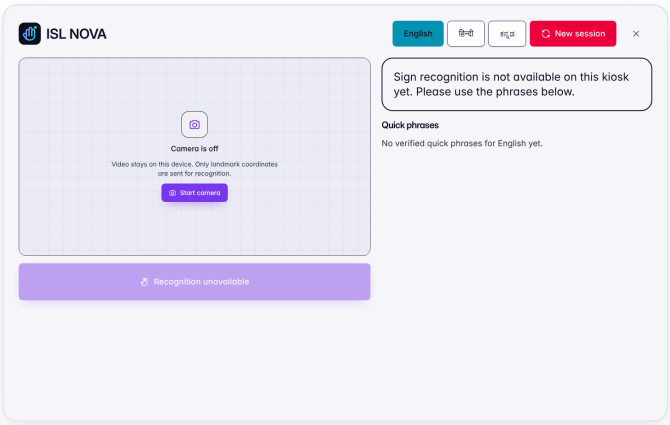
New-signer evaluation. Per-signer accuracy on consented recordings from outside INCLUDE.



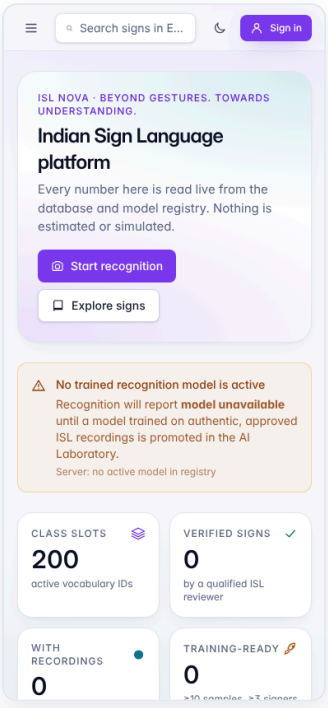
Communication. Composer with unverified-meaning warnings and speech.



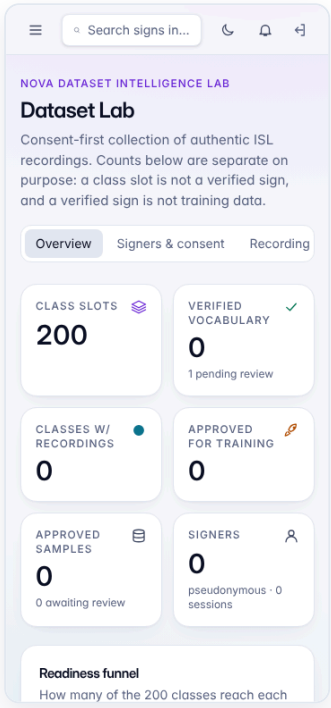
Dark theme. One token system drives both themes.



Kiosk mode. High contrast and large targets for public counters.



Phone, 390 px.



Dataset Lab, 360 px.

Design and accessibility

- ✔ Light theme by default, dark toggle or "match my device"
- ✔ Keyboard navigation, visible focus, skip link
- ✔ Labelled controls, image alt text, named links, landmarks
- ✔ High-contrast, reduced-motion and large-text modes
- ✔ No horizontal scroll on 11 routes at 360, 768 and 1280 px (tested)
- ✔ Every route has exactly one page heading and no unnamed buttons (tested)

Answer states a user can see

recognized · uncertain · unknown · insufficient input · model unavailable · camera unavailable

08 Testing, quality and security

96 automated tests, on Linux and Windows

60	17	19	0
backend + ML tests (pytest)	frontend unit + parity tests	end-to-end browser tests	failing in the final runs

Suite	What it covers	Result
Ruff lint	Python style and security rules	✅ pass
pytest (SQLite + PostgreSQL 16)	API, auth, RBAC, CRUD, consent, erasure, dataset, registry, inference states, gradients, leakage, open set	✅ 60 passed, 1 skipped
TypeScript strict + unit	Feature parity with Python, router, store, UI components, validator	✅ 17 passed
Playwright E2E	Sign-up, login, RBAC, search, CRUD, verification, consent → record → review, camera states, kiosk, themes, layouts, 3 tests with the real model	✅ 19 passed
Docker	Image build and container health check	✅ pass
Windows runner + owner's PC	Setup, start, 24/24 via local API, all suites	✅ all passed
Live deployment smoke	Health, CSP/HSTS, model, 24/24 via public API, UI at 1280 and 390 px	✅ all passed

Bugs the tests found, and we fixed

- **Windows video upload** failed: OpenCV cannot reopen an open temp file on Windows
- **GPU delegate at ~1 fps** on software WebGL → automatic CPU switch
- **Silent empty capture** → now an explicit *insufficient input*
- **Fake-camera path** must be absolute for Chromium
- **API docs blocked** by our own CSP → same-origin init script
- **PostgreSQL FK order** on signer erasure (found by CI)
- **Decimal** values from PostgreSQL SUM() broke JSON
- **DTW memory blow-up** → batched einsum formulation
- **MediaPipe timestamps** must be monotonic across videos
- **Mobile overflow** of 7–38 px on narrow screens

Security controls

Identity

PBKDF2-SHA256 password hashing, JWT with expiry, server-side RBAC for 5 roles, rate-limited login.

Web

Strict CSP, X-Frame-Options DENY, HSTS in production, no wildcard CORS, upload magic-byte and decode checks.

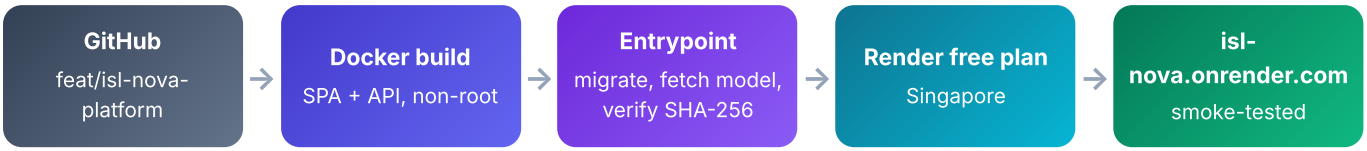
Data

No secrets in Git, path-safe storage, audited media access, checksummed models and migrations.

09 Deployment, setup and project structure

Live online, and one command on a Windows PC

Live deployment



The demo runs with no paid resources. The free plan sleeps when idle (first request ~1 minute) and resets its SQLite database on every restart, so the demo has no admin account and should not receive real signer recordings.

Run it on Windows

```
# 1. get the code
git clone https://github.com/nvaltryek-001/ISL-NOVA.git ; cd ISL-NOVA
git checkout feat/isl-nova-platform
# 2. one-time setup: venv, packages, frontend build, .env, database, trained model
powershell -ExecutionPolicy Bypass -File scripts\setup_windows.ps1
# 3. start: http://127.0.0.1:8000
powershell -ExecutionPolicy Bypass -File scripts\start_windows.ps1
# 4. download the INCLUDE landmarks (176 MB) and train locally
powershell -ExecutionPolicy Bypass -File scripts\train_windows.ps1 -Model all
# 5. tests
powershell -ExecutionPolicy Bypass -File scripts\test_windows.ps1 -E2E -WithModel
```

Prerequisites

Python 3.13, Node.js 22, Git. Optional: PostgreSQL 16, Docker Desktop.

Docker

`docker compose up --build` runs the app with PostgreSQL 16; `ISL_NOVA_MODEL_TAG` installs the model.

Verified

Setup, start, tests and training all passed on a Windows runner and on the owner's PC.

GitHub Actions workflows

Workflow	Purpose
`ci.yml`	Lint, tests on SQLite and PostgreSQL, frontend, E2E with the model, Docker, dependency audit
`include-data · include-train`	Download and extract INCLUDE; train, compare, verify and release models
`include-open-set · include-live-eval`	Unknown-sign test; virtual-camera replay through the Live page
`deploy-smoke · windows-local · publish-landmarks`	Live-site checks; Windows scripts; publish the landmark dataset

Folder structure

```
ISL-NOVA/
├── backend/
│   ├── app/                 API framework, routers, db, migrations, security, services
│   │   ├── routers/         core · signs · dataset · ml · personal
│   │   └── migrations/      0001_core.sql (versioned, checksummed)
│   └── tests/               API, RBAC, CRUD, consent, evaluation, inference tests
├── ml/
│   ├── isl_nova_ml/         contract · features · models · train · registry · inference
│   │                        extract · include_pipeline · metrics · synthetic
│   ├── configs/             lstm_baseline · gru · pooled-logreg · dtw-1nn
│   ├── runs/                settings for the staged Actions workflows
│   └── tests/               contract, gradients, leakage, registry, open set, parity
├── frontend/
│   ├── src/pages/           Dashboard, Live, Explorer, DatasetLab, AILab, Kiosk, …
│   ├── src/components/      design system, charts, CameraStage, icons
│   ├── src/lib/             api, router, query, store, features, landmarks
│   └── tests/               unit, e2e, live-eval, deploy smoke
├── dataset/
│   ├── vocabulary/          signs_master.csv (200), translations, references, categories
│   └── sources/include/     label map, official train/val/test lists, licence
├── docs/                    architecture, API, schema, model card, deployment, status
├── research/                results, open set, protocols, limitations, references
├── scripts/                 setup/start/test/train_windows.ps1, fetch_model, train_local
├── .github/workflows/       ci + data, train, evaluation, deploy, Windows pipelines
├── Dockerfile · docker-compose.yml · .env.example
└── README.md
```

API overview (84 operations)

Area	Examples
Core	`GET /health` · `POST /auth/login` · `GET /me`
Signs	`GET /signs` · `POST /signs` · `POST /signs/{id}/review`
Recognition	`POST /predict/sequence` · `GET /model/info`
Dataset	`POST /dataset/recordings` · `/consents` · `/versions`
AI Lab	`POST /experiments/{id}/run` · `/models/{id}/promote`
Evaluation	`GET /evaluation/metrics` · `POST /evaluation/runs`

Interactive documentation is served at `/api/docs`; the full contract is generated from the code into docs/API_CONTRACT.md.

10 Limitations, future work and conclusion

What is proven, what is not, and what comes next

Limitations, stated plainly

- ! Results are on INCLUDE's official split, **not signer-independent**
- ! **29 %** of unseen signs are still recognized as a known sign
- ! No test yet with **a real person at a webcam** or with new signers
- ! **0 / 200** signs verified by qualified ISL reviewers
- ! Isolated signs only; no grammar, translation or facial expressions
- ! Free demo sleeps when idle and loses its data on restart

Future work

- Consented recordings from ≥ 10 new signers for a true signer-independent test
- Stronger unknown-sign detection: energy scores, background class, embeddings
- TCN and Transformer models; feature ablations; multiple seeds
- Real-user studies with Deaf signers, with consent and a protocol
- Verified Hindi and Kannada meanings and regional variants
- Persistent hosting with managed PostgreSQL

Conclusion

ISL NOVA turns a genuine, licensed ISL dataset into a working, tested and deployed recognition platform. An LSTM trained on 3,370 real INCLUDE recordings recognises all 200 signs with **91.7 % top-1** on held-out videos, and the same model answers correctly through the live public API.

Just as important, the project knows its limits. Each answer carries a calibrated state; unknown-sign rejection, new-signer accuracy and real-webcam use are measured or marked as open; data collection is built around consent; and every number in this report can be traced to the run that produced it.

References

1. Sridhar, A., Ganesan, R. G., Kumar, P., Khapra, M. (2020). INCLUDE: A Large Scale Dataset for Indian Sign Language Recognition. ACM Multimedia. doi:10.1145/3394171.3413528
2. Joshi, A. et al. (2024). iSign: A Benchmark for Indian Sign Language Processing. Findings of ACL.
3. Hochreiter, S., Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation 9(8).
4. Guo, C. et al. (2017). On Calibration of Modern Neural Networks. ICML.
5. Hendrycks, D., Gimpel, K. (2017). A Baseline for Detecting Misclassified and Out-of-Distribution Examples. ICLR.
6. Lugaresi, C. et al. (2019). MediaPipe: A Framework for Building Perception Pipelines. arXiv:1906.08172.

Dataset attribution. INCLUDE by A. Sridhar, R. G. Ganesan, P. Kumar and M. Khapra (AI4Bharat, IIT Madras), CC BY 4.0. Landmark images in this report are derived from it. MediaPipe is © Google, used under its licence.